SEPARADOR 03 — FICHA TÉCNICA DEL SISTEMA

Documento Técnico de Homologación del sistema KONTROL. Contenido técnico real del producto, versión 1.0.0.

> La ficha técnica describe el aplicativo, el lenguaje, la base de datos, el monitoreo y auditoría, y el tipo de conexión con las plataformas, conforme a la exigencia de la Providencia Administrativa. Los datos se toman de la documentación técnica y de las evidencias de desarrollo de KONTROL.

A. Identificación

Aplicativo	**KONTROL** — Plataforma Administrativa Empresarial para Venezuela
Edición	Edición empresarial (multiempresa / monoempresa por licencia)
Versión exacta	1.0.0 (backend y frontend)
Build / liberación	Build automatizado por CI (GitHub Actions) para ramas `develop` y `main`; imagen Docker etiquetada por entorno
Fecha de liberación	2026 (candidata a homologación)
Propietario	KONTROL (producto)
Desarrollador	Equipo de desarrollo KONTROL (dvconsultores)
Proveedor	dvconsultores (publicador de las imágenes de contenedor de KONTROL)
Arquitectura	Cliente-servidor web: SPA (navegador) + API REST + base de datos relacional, empaquetada en contenedores
Tipo de implementación	Instalación por cliente en contenedores Docker (Compose); base de datos PostgreSQL externa por instalación

B. Arquitectura tecnológica

B.1 Componentes

Componente	Tecnología	Descripción
Frontend	React 19 + TypeScript 5.7 + Vite 6	Interfaz de usuario de página única (SPA), servida estáticamente por Nginx
Backend	Python 3.13 + FastAPI + Uvicorn	API REST y lógica de negocio; empaquetado en imagen Docker
API	REST / JSON sobre HTTPS	Prefijo `/api/v1`; especificación OpenAPI; más de 200 rutas
Base de datos	PostgreSQL (psycopg2)	Motor relacional externo por instalación; migraciones Alembic
Servidor web	Nginx (reverse proxy + SPA)	Sirve el frontend, proxy de `/api/v1`, headers de seguridad y TLS

Docker	Imágenes multi-stage, usuario no root	Imágenes `kontrol-frontend`, `kontrol-backend` y proxy Nginx

B.2 Contenedores y orquestación

- **frontend** (`kontrol-web`): sirve la SPA; healthcheck `/healthz`.
- **backend** (`kontrol-api`): API Uvicorn en el puerto 8000; aplica las migraciones automáticamente al arranque (`alembic upgrade head`); healthcheck `/api/v1/health/live`.
- **watchtower**: actualización automática de imágenes etiquetadas, con reinicio rodante opcional.
- **Red**: red bridge `kontrol-net` de nombre fijo para que el proxy de entrada pueda enrutar a `kontrol-web:80` y `kontrol-api:8000` sin exponer puertos.
- **Base de datos**: externa (cada cliente la suya), alcanzada por el backend mediante `KONTROL_DB_URL`.

B.3 Sistema operativo soportado y servidor

- **Servidor**: Linux (imágenes base Debian slim y Alpine); cualquier host con motor Docker compatible.
- **Cliente**: navegadores web modernos de escritorio (Chrome/Edge, Firefox y Safari en sus versiones vigentes).
- **Navegadores compatibles**: la SPA se compila para navegadores modernos con soporte ES2020+.

B.4 Esquema de comunicaciones

- El navegador se comunica con el frontend servido por Nginx y con la API a través de `/api/v1` mediante HTTPS.
- El backend se comunica con PostgreSQL mediante el driver `psycopg2` (conexión TLS habilitable por configuración del motor).
- El backend mantiene comunicación saliente opcional con el servidor central de licencias (contrato REST v1) para la sincronización de la licencia y los entitlements por módulo.
- Todo intercambio de datos con el exterior se realiza sobre HTTPS/TLS a través del reverse proxy.

B.5 Servicios internos

Servicio	Función
Licenciamiento local y central	Verificación de licencia firmada (Ed25519), modos de acceso, sincronización central, entitlements por módulo, renovación con comprobante
Identidad y acceso	Autenticación (Argon2 + JWT), sesiones, perfiles y permisos
Organización	Instalación/tenant, empresa, sucursal, establecimiento, almacén y ubicación
Maestros	Terceros, productos, servicios y catálogos
Núcleo fiscal	Fuentes legales, reglas tributarias versionadas, series de numeración, números de control, cálculos y perfiles fiscales
Ventas y cuentas por cobrar	Facturación, notas de crédito/débito, multimoneda, formas de pago, CxC

Compras y cuentas por pagar	Compras, retenciones, CxP
Inventario y logística	Existencias, kardex, transferencias, conteos y ajustes
Tesorería	Cajas, bancos, ingresos/egresos, conciliación, IGTF
Regulación y traslados	Permisos, guías de traslado (SICA/SIGESAI/SACS), evaluación y bloqueo de despacho
Reportes y auditoría	Dashboard, centro de reportes, exportación auditada y línea de tiempo
Homologación y liberación	Actos formales, versiones fiscales, gates y liberación de versión

C. Lenguajes y tecnologías

C.1 Backend

Componente	Versión
Lenguaje	Python 3.13 (producción); ≥ 3.11 en estaciones de desarrollo
Framework web	FastAPI 0.141.1
Servidor ASGI	Uvicorn 0.32.1
Validación / configuración	Pydantic 2.10.4 · pydantic-settings 2.7.0
ORM	SQLAlchemy 2.0.36
Migraciones	Alembic 1.14.0 (11 migraciones, head `0011_homologacion`)
Driver de base de datos	psycopg2-binary 2.9.10
Hash de contraseñas	Argon2 (argon2-cffi 23.1.0)
Tokens / firma	PyJWT 2.13.0 · cryptography 46.0.6 (Ed25519 para licencias)
Validación de correo	email-validator 2.2.0 · python-multipart 0.0.31

C.2 Frontend

Componente	Versión
Lenguaje	TypeScript 5.7 (modo estricto)
UI	React 19
Build	Vite 6.4.3
Estilos	Tailwind CSS 4
Enrutado	React Router 8.3
Datos	TanStack Query 5.62 · TanStack Table 8.20
Formularios	React Hook Form 7.54 · Zod 3.24

Iconografía	Lucide React 0.469
Calidad	ESLint 9 · Prettier 3.4 · Vitest 3.2 · Playwright 1.62

C.3 Infraestructura

Componente	Tecnología / versión
Motor de base de datos	PostgreSQL (driver psycopg2 2.9.10); base externa por instalación; respaldo de referencia con imagen `postgres:18-alpine`
Servidor web	Nginx (imagen `nginx:stable-alpine`) — SPA, reverse proxy y TLS
Contenedores	Docker (Dockerfile multi-stage por servicio) + Docker Compose
CI / build	GitHub Actions — builds de imagen para `develop` y `main`
Actualización	Watchtower (etiquetado de imágenes, reinicio rodante opcional)
Componentes de generación fiscal	Núcleo fiscal (007): `LegalSource`, `TaxRuleVersion`, `FiscalNumberSeries`, `ControlNumberBatch` / `ControlNumber`, `TaxCalculation`, `TaxProfile`

D. Base de datos

D.1 Motor y versión

KONTROL utiliza **PostgreSQL** como motor relacional. La base de datos es **externa por instalación** (cada cliente dispone de la suya) y se conecta mediante `KONTROL_DB_URL` con un **usuario restringido** (no superusuario). El despliegue de referencia y el respaldo usan imágenes PostgreSQL modernas (referencia `postgres:18-alpine`); el driver es `psycopg2` 2.9.10 y el ORM SQLAlchemy 2.0.36.

D.2 Estructura general y modelo de almacenamiento

- Esquema relacional normalizado, gestionado por migraciones **Alembic** (versionadas y reproducibles; head `0011_homologacion`).
- Identificadores UUID en las entidades; fechas con zona horaria (`DateTime(timezone=True)`).
- Modelo de datos: `Installation → Tenant → Company → Branch → Establishment → Warehouse → Location`, con aislamiento por empresa (ADR-011).
- Datos fiscales versionados: reglas tributarias con vigencia (`TaxRuleVersion`), fuentes legales con hash (`LegalSource`).
- Los eventos de auditoría son de solo agregar (*append-only*) (`audit_events`).

D.3 Identificación, numeración y correlativos de documentos fiscales

- **Series de numeración fiscal** (`FiscalNumberSeries`): por empresa y establecimiento, con tipo de documento, prefijo y número siguiente.
- **Asignación atómica**: se usa control de concurrencia optimista (`lock_version`) para evitar duplicados o saltos no controlados en la asignación de correlativos.

- **Números de control** (`ControlNumberBatch` / `ControlNumber`): lotes de imprenta autorizada, con rango y estado individual (`AVAILABLE / RESERVED / USED / VOID / LOST / DAMAGED`) y referencia al documento que los consume.
- La **emisión real** de documentos y su numeración definitiva corresponden al núcleo fiscal (spec 007) y quedan sujetas al proceso de homologación de la versión fiscal correspondiente.

D.4 Conservación de datos, restricciones e integridad referencial

- **Integridad referencial**: claves foráneas y restricciones de unicidad gestionadas por PostgreSQL.
- **Conservación**: los documentos fiscales y los eventos de auditoría se conservan de forma persistente; no se eliminan registros fiscales por operación normal (ver Separador 04).
- **Trazabilidad**: cada operación se vincula a tenant, empresa, usuario, sesión, request y fecha/hora mediante el registro de auditoría.
- **Protección frente a eliminación/modificación**: la historia fiscal es de solo agregar a nivel de aplicación; los permisos de base de datos se restringen (usuario de aplicación sin privilegios de borrado administrativo sobre tablas fiscales en producción).

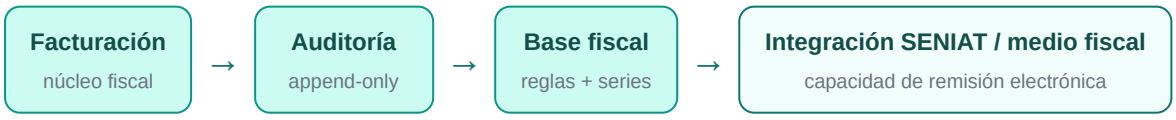
D.5 Respaldos y restauración

Elemento	Descripción
Respaldo	Script `scripts/backup.sh` : `pg_dump` de la base externa (en contenedor PostgreSQL), respaldo de volúmenes de storage y estado de licencia
Cifrado	Cifrado opcional del respaldo con AES-256-CBC (PBKDF2) mediante clave de instalación
Restauración	Script `scripts/restore.sh` para recuperar el respaldo
Volúmenes	`/data/storage` (archivos), `/data/license-state` (identidad de instalación y clave), `/data/backups` (respaldos)

E. Arquitectura diagramada



Esquema principal: Usuario → Frontend → Backend/API → PostgreSQL



Servicios asociados: Facturación → Auditoría → Base fiscal → Integración SENIAT / medio fiscal

> **Tipos de conexión con las plataformas**
>
> La ficha técnica indica de forma expresa el tipo de conexión en el **Separador 05** de este expediente (protocolo HTTPS/TLS, API REST, autenticación, serialización JSON, reintentos, registro de

transmisiones, contingencia y cifrado), sin afirmar protocolos concretos que SENIAT no haya especificado para la integración a evaluar.